ESPRIT · IA FOR SOFTWARE ENGINEERING

Documentation individuelle

Usage de l'IA générative, prompts et justification du modèle

Khadija Ben Ayed

M2 · Irrigation et alertes météo

Projet	Zitouna AI — plateforme d'IA pour les petits oléiculteurs tunisiens
Rôle transversal	Backend et intégration des API
Classe	5SAE10
Encadrant	Khemiri Akram
Année universitaire	2026 – 2027

1. Contexte et rôle

Dans le projet Zitouna AI, j'ai porté le module M2 · Irrigation et alertes météo de bout en bout : données, modèle, service FastAPI, intégration dans l'application et écran. Type d'IA : Séries temporelles · réseau de neurones récurrent (LSTM). Rôle transversal : Backend et intégration des API.

2. Outils d'IA générative utilisés

Outil	Nature	Usage
Claude Code (Anthropic)	Assistant de développement connecté au dépôt	Intégration de M2, backend, frontend, tests, revue de sécurité, rapport, présentation, vidéo
Gemini, Groq	LLM intégrés dans l'application (M5)	Réponses de l'assistant en derja
Bandit, npm audit, JUnit, Vitest, pytest	Contrôle des propositions de l'IA	Sécurité et tests automatiques
Gemini 3.7 Flash (Google)	LLM dédié à l'explication de M2	Explique le plan d'irrigation en français ou en derja

3. Méthode de travail avec l'IA

Chaque échange suit la même boucle : une demande précise avec ses contraintes, une proposition de l'IA, une exécution réelle (test, appel d'API, capture), puis une correction si le résultat ne convient pas. Aucun chiffre n'est gardé sans avoir été mesuré.

Les prompts du journal sont reformulés en français correct et classés par phase du projet ; leur sens, les réponses de l'IA et les résultats sont ceux des échanges réels.

4. Journal des prompts et des interactions

Date	Phase	Prompt (reformulé)	Réponse / action de l'IA	Vérification
17/10/2026	Analyse	Vérifie que le projet démarre correctement et que tous les tests passent, puis signale-moi les erreurs.	Lancement des 9 conteneurs et des tests un par un dans Docker	Journaux relus ; installations relancées sur réseau lent
17/10/2026	Modèle	Réalise le module M2 avec un vrai modèle de prévision, entraîné et évalué sur des données réelles.	Entraînement du LSTM sur ERA5, service FastAPI, écran d'irrigation	8 tests ; comparaison avec Open-Meteo à Sfax
17/10/2026	Données	Remplace les valeurs simulées par une vraie API météo, avec une solution de repli en cas de panne.	Branchement d'Open-Meteo, cache d'une heure, repli explicite	Appel réel ; test « panne de la météo »
17/10/2026	Intégration	Supprime Swagger et vérifie que chaque module utilise un vrai modèle, sans données simulées.	Suppression de springdoc, page d'état, entraînement réel de M1 et M6	Page /api/status : 6 « Modèle réel »
17/10/2026	Sécurité	Crée un seul compte administrateur et des comptes agriculteurs, chacun redirigé vers son espace.	Administrateur unique, routes /api/admin protégées	Agriculteur → 403, administrateur → 200
17/10/2026	Interface	Réorganise le frontend pour qu'il soit clair,	Barre latérale, palette olive et or, irrigation sur 7	15 tests Angular, contrôle de chaque

Date	Phase	Prompt (reformulé)	Réponse / action de l'IA	Vérification
		cohérent et professionnel.	ou 14 jours	page
18/10/2026	Tests	Vérifie que tous les modules de la vidéo de démonstration utilisent les vrais modèles.	Lecture des prédictions enregistrées pendant la vidéo	7 appels réels, aucun en mode de repli
18/10/2026	Qualité	Compare le projet à la grille d'évaluation et complète ce qui manque, dont la revue de sécurité.	Revue de sécurité, correction de 2 alertes Bandit, chapitre IA générative	Tests relancés ; Bandit sans alerte
18/10/2026	Documentation	Donne-moi les commandes de chaque série de tests pour que je réalise moi-même les captures.	Commandes PowerShell pour chaque série de tests	Captures prises par moi : 154 tests réussis
18/10/2026	Intégration	Ajoute à M2 un bouton « Expliquer avec l'IA » qui utilise le LLM le mieux adapté à ce module.	LLM dédié : Gemini 3.7 Flash (la meilleure derja sur le tableau jour par jour parmi cinq modèles comparés) ; le reste de la chaîne Gemini/Groq sert de secours	Test réel : explication rédigée par gemini-3.7-flash en 3,1 s
18/10/2026	Qualité	Le LLM dédié à M2 ne répond pas : trouve la cause dans les journaux et corrige-la.	Gemini 2.5 Flash retiré par Google (404) : remplacé par Gemini 3.8 Flash ; délai porté à 8 s	Clé testée avant le changement ; explication en derja par gemini-3.8-flash
18/10/2026	Optimisation	Compare cinq LLM sur le même plan d'irrigation réel, en français et en derja, et choisis pour M2 un modèle différent de ceux des autres modules.	Gemini 3.7 Flash retenu ; GPT-OSS 20B écarté (derja fautive), Gemma 4 et ALLaM 2 en échec	Mesures : 3,0 s pour 3.7 Flash, 4,2 à 5,0 s pour 3.8 Flash ; test : six modules, six LLM différents

5. Justification du modèle d'IA choisi

Modèle comparé	Résultat sur les données de test
Même jour l'an dernier (référence)	MAE 1,06 mm/j
Persistance sur 7 jours	MAE 0,80 mm/j
Prophet (dates communes)	MAE 0,666 mm/j
LSTM global (modèle retenu)	MAE 0,71 mm/j · 0,657 sur les dates communes

Le LSTM a l'erreur la plus faible : 33 % de moins que la référence, et légèrement mieux que Prophet sur les mêmes dates. Ses poids sont exécutés en NumPy dans le service (écart avec PyTorch de $1{,}6 \times 10^{-6}$ mm/j) : l'image Docker reste légère (327 Mo) et la réponse est immédiate. Le besoin en eau suit ensuite la formule FAO-56, avec des alertes gel (≤ 0 °C) et canicule (≥ 40 °C) issues des prévisions Open-Meteo.

6. Gains de productivité

Tâche	Apport de l'IA	Limite constatée
Intégration d'un module (service, API, écran)	Code et tests produits dans la même séance	Relecture de chaque fichier
Diagnostic d'erreurs Docker et Maven	Analyse des journaux et contournements proposés	Plusieurs essais parfois nécessaires
Tests automatiques	Cas limites écrits rapidement (panne météo, alertes)	Un test peut être ignoré si l'environnement est faux

Tâche	Apport de l'IA	Limite constatée
Documentation et démonstration	Rapport, captures et vidéo produits à partir des résultats réels	Contenu validé par moi

7. Limites et bonnes pratiques retenues

- Donner des consignes précises et des contraintes explicites (« valeurs réelles, pas de mock ») améliore nettement le résultat.
- Ne jamais accepter un résultat sans l'exécuter : chaque proposition a été vérifiée par un test, un appel réel ou une capture.
- Fournir le contexte (capture d'écran, message d'erreur complet) accélère le diagnostic.
- Ne jamais partager de secret avec l'IA ni dans une capture : une clé d'API visible dans une capture a dû être régénérée.
- L'IA se trompe parfois (commandes non compatibles avec PowerShell, chemins de modèles) : garder un esprit critique.

8. Conclusion

L'IA générative a accéléré l'intégration de M2 et de la plateforme, du code aux tests et à la documentation. Son apport dépend de la précision des demandes et de la vérification systématique de ses propositions : elle m'a fait gagner du temps sans remplacer mon jugement.